



Universidade de Brasília

Departamento de Ciência da Computação

CIC0199 - Teoria e Aplicação de Grafos - Turma 01 - 2026.1

Prof. Dr. Dúbio Leandro Borges.

Alunos: Daniel Florencio Hollenbach, matrícula 241020859,

Mateus Valério, matrícula 190035161 e

Maylla Krislainy, matrícula 190043873.

Trabalho 02 - Problema de Alocação Aluno-Projeto

1) Introdução

Este trabalho implementa uma solução para o Problema de Alocação Aluno-Projeto (*Student-Project Allocation* — SPA), com base em Abraham, Irving e Manlove (2007). O objetivo é alocar até 200 alunos em 50 projetos financiados, respeitando as preferências de cada aluno, a nota mínima exigida por cada projeto e o número de vagas disponíveis.

A solução foi implementada em Python como um pacote spa e executada em notebook Jupyter. O processo se divide em duas etapas: um emparelhamento estável inicial gerado por uma variação do Gale-Shapley orientada aos alunos (SPA-Student com substituição por mérito), seguido de dez iterações de melhoria por caminhos aumentantes, que tentam alocar os alunos que ficaram sem projeto na primeira etapa.

2) Modelagem de dados de Entrada

O arquivo de entrada `entradaProj2.26TAG.txt` é estruturado em duas seções distintas, conforme a especificação do projeto. A primeira descreve os projetos disponíveis enquanto a segunda descreve os alunos candidatos. Durante essa etapa, foram detectadas duas categorias de irregularidades nos dados fornecidos: referências a projetos inexistentes nas listas de preferência de alguns alunos e preferências duplicadas em algumas listas de alunos. Ambos os casos não foram considerados durante a criação do grafo bipartido e durante as iterações dos caminhos M-aumentantes. Porém, eles são processados pelo algoritmo de emparelhamento.

3) Emparelhamento Estável Inicial

3.1) Motivação e Escolha do Algoritmo

O problema proposto neste trabalho consiste em alocar, de forma impessoal e competitiva, até 200 alunos em até 50 projetos disponíveis, respeitando capacidades, notas mínimas exigidas e as preferências declaradas por cada parte. Dado que ambos os lados do problema possuem restrições e preferências, alunos ordenam seus projetos de interesse e projetos possuem critério mínimo de nota, a solução naturalmente se enquadra no modelo de emparelhamento

estável, conforme formalizado em Abraham, Irving e Manlove (2007) para o Student-Project Allocation (SPA).

Dentre as duas variações apresentadas no artigo de referência, SPA-Student e SPA-Lecturer, optou-se pela implementação do SPA-Student, com uma adaptação por mérito. A justificativa foi essencialmente prática e pedagógica: o algoritmo SPA-Student é orientado a propostas partindo dos alunos, o que reflete de maneira mais direta a dinâmica do problema (alunos candidatam-se a projetos), e sua lógica de execução é mais próxima do algoritmo Gale-Shapley clássico, já estudado na disciplina.

Na variação implementada, as propostas partem dos alunos em ordem crescente de matrícula, garantindo execução determinística. Cada aluno propõe sequencialmente aos projetos em sua lista de preferência. Um projeto aceita o aluno se: (a) a nota agregada do aluno atende à nota mínima exigida pelo projeto; e (b) há vaga disponível. Caso o projeto esteja cheio, mas o candidato possua nota estritamente maior que o pior aluno já alocado, o pior aluno é expulso e retorna à fila de livres, mecanismo de substituição por mérito. O algoritmo termina quando não há mais alunos livres com propostas pendentes.

A verificação formal de estabilidade do emparelhamento gerado foi realizada por meio da função `is_stable_matching`, implementada separadamente com auxílio de LLM (documentado em `prompts_ai.md`), confirmando que nenhum par (aluno, projeto) bloqueante existe no resultado: todo aluno não alocado foi rejeitado por todos os seus projetos de preferência, seja por nota insuficiente ou por haver candidatos de maior nota já ocupando todas as vagas.

3.2) Limitação da Abordagem e Adequação do SPA-Lecturer

Embora o emparelhamento M_0 seja formalmente estável, nenhum par bloqueante existe, ele apresenta uma limitação crítica em relação à especificação do projeto: a especificação exige que cada projeto tenha ao menos 1 aluno alocado, e esse requisito não foi atingido. Vários projetos permanecem sem nenhum aluno ao final do SPA-Student.

Nesse contexto, torna-se pertinente observar que a variação SPA-Lecturer teria sido mais adequada para este cenário. Nela, as propostas partem dos professores (orientadores dos projetos), que iterativamente oferecem vagas a alunos elegíveis ainda não alocados em projetos de sua cota. Essa orientação garante naturalmente que projetos com vagas disponíveis e alunos compatíveis sejam preenchidos de forma prioritária, maximizando o número de projetos com ao menos um aluno alocado, objetivo central da especificação.

A escolha do SPA-Student, portanto, priorizou a perspectiva dos alunos (garantindo a cada alocado o melhor projeto possível), ao custo de deixar projetos sem candidatos quando o conjunto de alunos elegíveis é escasso. Uma implementação futura com SPA-Lecturer, ou uma estratégia híbrida, SPA-Student para o matching estável inicial, seguido de uma fase de preenchimento orientada a projetos, poderia atender de forma mais completa aos requisitos da especificação.

4) Iterações de Melhoria por Caminhos Aumentantes

O emparelhamento estável M_0 , obtido na fase anterior pelo SPA-Student, é utilizado como ponto de partida para uma segunda fase de ampliação do matching, através de **10 iterações** de busca por caminhos M-aumentantes sobre o grafo bipartido G .

A lógica de cada iteração é a seguinte: os alunos livres são ordenados por matrícula, garantindo execução determinística. Para o primeiro aluno livre da fila, realiza-se uma busca em largura (BFS) por um caminho M-aumentante, isto é, um caminho alternado que parte do aluno livre por uma aresta fora do matching (aluno $\rightarrow$ projeto), e, se o projeto encontrado estiver cheio, prossegue por uma aresta dentro do matching (projeto $\rightarrow$ aluno alocado), alternando até alcançar um projeto com vaga disponível. Quando encontrado, o caminho é aumentado: o status de cada aresta do caminho é invertido, o que incorpora um novo par ao matching e libera o aluno anteriormente alocado na outra extremidade. O grafo é plotado antes de cada augmentação, exibindo em cores distintas a proposta ativa, o matching corrente e as rejeições acumuladas. Ao fim da iteração, a lista de alunos livres é atualizada para a rodada seguinte. Caso nenhum caminho aumentante seja encontrado, o matching já é máximo e as iterações restantes não produzem alterações.

É importante destacar que a busca por caminhos aumentantes não preserva a propriedade de estabilidade do emparelhamento. Nesta fase prioriza-se a maximização da cardinalidade do matching, alocando o maior número possível de alunos, sem considerar as preferências das partes ao montar os novos pares. Essa perda de estabilidade foi verificada formalmente pela função *is_stable_matching* ao final das 10 iterações, confirmando que o matching final contém pares bloqueantes.

5) Grafo Bipartido e Visualização

O problema SPA foi modelado como um grafo bipartido $G = (A \cup P, E)$, onde A representa os 200 alunos e P os 50 projetos. Uma aresta (a, p) é criada sempre que o projeto p aparece na lista de preferências do aluno a , carregando o atributo *preferência* $\in \{1, 2, 3\}$ para indicar a posição daquele projeto na ordem declarada pelo aluno. A nota mínima não filtra arestas nesta etapa, essa verificação acontece durante a execução do algoritmo, mantendo o grafo fiel ao espaço completo de preferências declaradas.

A representação como grafo bipartido explícito, usando NetworkX, foi escolhida por facilitar diretamente a busca por caminhos alternados via BFS, que é o núcleo das iterações de melhoria. Cada nó de projeto também armazena a lista de todos os alunos que o indicaram, usada para calcular o ranking na matriz final.

A visualização a cada iteração usa três cores de aresta:

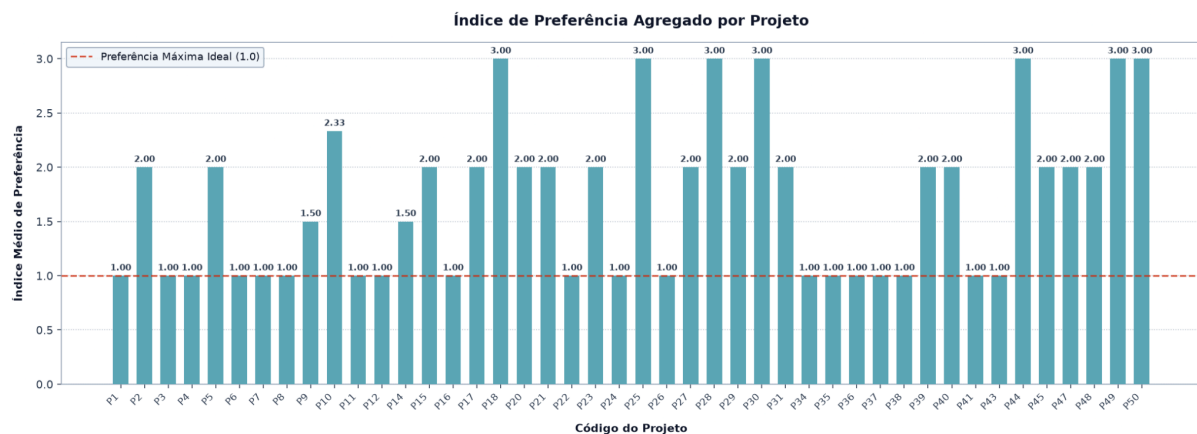
- **Azul** — par alocado no matching atual
- **Vermelho tracejado** — proposta rejeitada (nota insuficiente ou projeto cheio)
- **Cinza** — aresta ainda não avaliada naquela iteração

O grafo é plotado antes de cada augmentação, mostrando a proposta sendo avaliada e não apenas o resultado, tornando o raciocínio do algoritmo visível a cada passo, conforme pedido pelo enunciado. As imagens de cada iteração são geradas e salvas automaticamente na pasta *output/*, com o formato *iteracao_N.png* (ex: *iteracao_0.png* para o estado inicial pós-GS, *iteracao_1.png* para a primeira melhoria, e assim por diante).

6) Matriz de Emparelhamento e Índice de Preferência

6.1) Índice de preferência por projeto

Para cada projeto, calcula-se a média da posição de preferência dos alunos alocados (1ª, 2ª ou 3ª escolha). Índice 1,0 significa que todos os alocados tinham aquele projeto como primeira opção; quanto maior o índice, menor a satisfação dos alunos com aquela alocação. O gráfico de barras gerado permite comparar essa métrica entre todos os projetos. Projetos com índice alto ou sem alunos alocados geralmente refletem a exigência de nota mínima elevada com poucos candidatos qualificados.



6.1) Matriz de emparelhamento final

Registra, para cada aluno alocado, o projeto emparelhado, sua posição na fila de candidatos do projeto (ordenada por nota decrescente) e a posição do projeto na sua lista de preferências. Ter as duas classificações juntas permite enxergar o resultado de cada lado do par, se um aluno foi alocado em sua primeira escolha e é o candidato mais bem avaliado do projeto, o emparelhamento foi ótimo para ambos; se foi alocado em sua terceira opção e ocupa a última posição na fila, o resultado foi ruim para os dois lados. A matriz completa é exportada em *output/matching_matrix.csv*.

7) Conclusão

A implementação alocou 63 dos 200 alunos candidatos, resultado que reflete uma limitação dos próprios dados de entrada: grande parte dos projetos exige nota mínima 5, enquanto a maioria dos alunos possui nota 3 ou 4, restringindo severamente o conjunto de pares compatíveis. Dentro dessas restrições, o algoritmo encontrou o matching máximo estável possível.

A variação SPA-Student com substituição por mérito cumpriu bem seu papel na primeira fase, garantindo que cada aluno alocado obtivesse o melhor projeto possível dado o cenário. No entanto, como discutido na Seção 3.2, essa orientação aos alunos teve como custo deixar vários projetos sem nenhum candidato, o que não atende plenamente à especificação. A fase de caminhos aumentantes conseguiu ampliar o matching, mas ao custo de perder a propriedade de estabilidade, trade-off esperado e verificado formalmente.

Uma abordagem mais completa passaria pelo uso do SPA-Lecturer ou de uma estratégia híbrida, que garantisse ao menos um aluno por projeto antes de otimizar as preferências.

8) Referências Bibliográficas

Material disponibilizado pelo próprio professor na plataforma Aprender3. Disponível em: <<https://aprender3.unb.br/mod/assign/view.php?id=1578936>>.

Abraham, D.J.; Irving, R.W.; Manlove, D.F. Two algorithms for the Student-Project Allocation problem. *Journal of Discrete Algorithms*, v. 5, n. 1, p. 73–90, 2007.